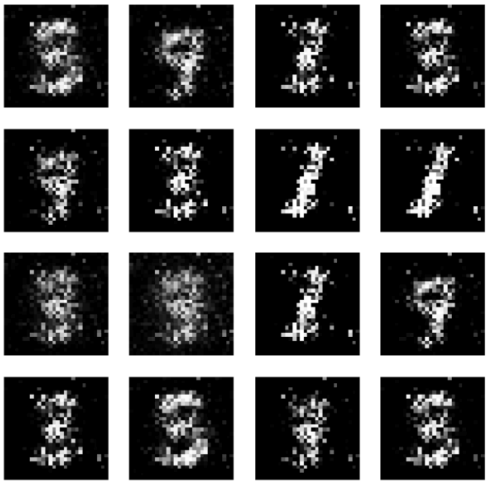
<u>EXP 3:</u> Implement a Generative Adversarial Network (GAN) to generate synthetic images or tabular data.
Dataset: <u>MNIST (for images)</u> or Synthetic Financial Transactions.

CODE:

```
# ==============================
# MNIST GAN - Single Block Code
# ==============================

# Install required packages
!pip install torch torchvision matplotlib

# Imports
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

# Device configuration
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# Data preprocessing and loading
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    transform=transform,
    download=True
)

dataloader = DataLoader(train_dataset, batch_size=128, shuffle=True)

# Generator model
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(100, 256),
```

```
            nn.ReLU(True),
            nn.Linear(256, 512),
            nn.ReLU(True),
            nn.Linear(512, 1024),
            nn.ReLU(True),
            nn.Linear(1024, 28 * 28),
            nn.Tanh()
        )

    def forward(self, z):
        return self.model(z)

# Discriminator model
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(28 * 28, 1024),
            nn.LeakyReLU(0.2),
            nn.Linear(1024, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.model(x)

# Initialize models
generator = Generator().to(device)
discriminator = Discriminator().to(device)

# Loss and optimizers
criterion = nn.BCELoss()
g_optimizer = optim.Adam(generator.parameters(), lr=0.0002)
d_optimizer = optim.Adam(discriminator.parameters(), lr=0.0002)

# Training loop
epochs = 20

for epoch in range(epochs):
    for real_images, _ in dataloader:
        real_images = real_images.view(-1, 28 * 28).to(device)
        batch_size = real_images.size(0)

        real_labels = torch.ones(batch_size, 1).to(device)
        fake_labels = torch.zeros(batch_size, 1).to(device)

        # Train Discriminator
```

```
        outputs = discriminator(real_images)
        d_loss_real = criterion(outputs, real_labels)

        noise = torch.randn(batch_size, 100).to(device)
        fake_images = generator(noise)

        outputs = discriminator(fake_images.detach())
        d_loss_fake = criterion(outputs, fake_labels)

        d_loss = d_loss_real + d_loss_fake
        d_optimizer.zero_grad()
        d_loss.backward()
        d_optimizer.step()

        # Train Generator
        outputs = discriminator(fake_images)
        g_loss = criterion(outputs, real_labels)

        g_optimizer.zero_grad()
        g_loss.backward()
        g_optimizer.step()

    print(f"Epoch [{epoch+1}/{epochs}] | D Loss: {d_loss.item():.4f} |
G Loss: {g_loss.item():.4f}")

# Generate and visualize synthetic images
generator.eval()
noise = torch.randn(16, 100).to(device)
generated_images = generator(noise).view(-1, 28, 28).cpu().detach()

fig, axes = plt.subplots(4, 4, figsize=(6, 6))
for i, ax in enumerate(axes.flatten()):
    ax.imshow(generated_images[i], cmap="gray")
    ax.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT:

```
Epoch [1/20], D Loss: 0.0165, G Loss: 8.5267
Epoch [2/20], D Loss: 1.4034, G Loss: 4.0903
Epoch [3/20], D Loss: 1.1779, G Loss: 3.3336
Epoch [4/20], D Loss: 0.2754, G Loss: 2.1744
Epoch [5/20], D Loss: 0.0261, G Loss: 9.2609
Epoch [6/20], D Loss: 0.9596, G Loss: 1.3719
Epoch [7/20], D Loss: 0.5424, G Loss: 1.6470
Epoch [8/20], D Loss: 1.0349, G Loss: 1.9811
Epoch [9/20], D Loss: 0.1999, G Loss: 4.0076
Epoch [10/20], D Loss: 0.6988, G Loss: 6.3794
Epoch [11/20], D Loss: 0.3461, G Loss: 3.0926
Epoch [12/20], D Loss: 0.8838, G Loss: 4.4472
Epoch [13/20], D Loss: 0.2569, G Loss: 3.6126
Epoch [14/20], D Loss: 0.4766, G Loss: 6.6975
Epoch [15/20], D Loss: 1.3687, G Loss: 3.1234
Epoch [16/20], D Loss: 0.9458, G Loss: 6.9430
Epoch [17/20], D Loss: 0.2620, G Loss: 3.5639
Epoch [18/20], D Loss: 0.3890, G Loss: 3.3517
Epoch [19/20], D Loss: 0.5364, G Loss: 3.6832
Epoch [20/20], D Loss: 0.4570, G Loss: 3.5807
```